МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 1

Варіант 6

Основи функціонального програмування у Python

Виконав:
Студент 2 курсу
групи ФЕП-22
Мазан Давид

Перевірив:
Аспірант
Остріков Данило Андрійович

Львів - 2026

Мета роботи

Ознайомитися з принципами функціонального програмування на Python: чистими функціями та референтною прозорістю; реалізувати обробку посилок без мутації вхідних даних; передавати політики тарифу, надбавки й прийнятності через typing.Callable; відокремити обчислювальне ядро від введення та виведення; перевірити реалізацію модульними тестами.

Теоретичні відомості

Чиста функція за однакових аргументів повертає однаковий результат і не змінює зовнішній стан та вхідні об’єкти. Референтна прозорість означає, що виклик функції можна замінити його результатом, не змінюючи поведінки програми.

Побічними ефектами є друк, робота з файлами чи мережею, зміна глобальних змінних або мутація переданих структур. Тому в проєкті обчислення зосереджені в core.py, а запуск і друк результату винесені в app.py.

typing.Callable дає змогу передавати функції як параметри. Фабрика make_delivery_processor приймає критерій прийнятності посилки, політику тарифу, політику надбавки та постачальник часу й повертає налаштований обробник.

Постановка завдання

Для варіанта 6 необхідно розробити систему розрахунку вартості доставки. Вхідний запис містить ідентифікатор посилки, вагу, відстань, тип доставки та ознаку оплати. Обробник має приймати лише допустимі оплачені посилки, обчислювати базовий тариф і надбавки, формувати нові записи результатів та підсумковий звіт. Необхідно забезпечити параметризацію правил через Callable, відсутність мутації вхідних даних, ізоляцію I/O та pytest-тести.

Хід роботи

1. Структура проєкту

Створено проєкт lab01 з модулем core.py для типів і чистих функцій, app.py для запуску та відображення звіту, каталогом tests із файлом test_core.py, а також README.md і requirements.txt. Для проєкту використано pytest.

2. Реалізація чистого ядра

У core.py описано типи ParcelInput, ProcessedParcel і DeliveryReport, а також Callable-псевдоніми для політик. Вхідні посилки читаються без змін; результат кожної обробки формується як новий словник із полями базової вартості, надбавки, повної вартості й часу обробки.

- calculate_base_tariff обчислює тариф за вагою та відстанню з множником типу доставки.

- calculate_surcharge додає фіксовану надбавку для важкої посилки та відсоткову надбавку для віддаленої доставки.
- with_delivery_totals створює новий запис обробленої посилки.
- filter_by_delivery_type повертає новий список посилок вибраного типу.
- process_parcels_pure обробляє партію з явними параметрами та повертає агрегований звіт.
- make_delivery_processor створює функцію-обробник, отримуючи політики прийнятності, тарифу, надбавки і часу через Callable.

3. Оболонка та демонстраційний запуск

У app.py визначено набір тестових посилок і конкретні політики: перевіряється оплата, допустима вага та додатна відстань; тарифна політика використовує ставки 12 грн/кг і 1,5 грн/км з множниками типу доставки; надбавка розраховується за поріг ваги 10 кг і віддаленої відстані 250 км. Додатково композиція множника 1,05 і пакувальної надбавки 15 грн застосовується лише тоді, коли первинна надбавка більша за нуль. Виведення підсумків зосереджено у render_report.

Під час запуску app.py оброблено три посилки. До звіту потрапили посилки №101, №102 і №105; результат відображає базові тарифи, надбавки, повну вартість кожної посилки, сумарний базовий тариф 1 249,47 грн, надбавки 274,06 грн і загальну виручку 1 523,53 грн. Ці значення наведені за демонстраційним запуском на скриншоті.

4. Тестування

У test_core.py реалізовано п’ять тестів: референтна прозорість, відсутність зміни вхідних посилок, перевірка Callable-політик і фабрики обробника, композиція функцій та множника, фільтрація за типом доставки без побічних ефектів.

За скриншотом запуску pytest усі 5 тестів завершилися зі статусом PASSED (5 passed, 0.02s), процес завершився з кодом 0. Окремий тестовий сценарій фабрики перевіряє звіт зі значеннями: 2 посилки, базовий тариф 1 265,00 грн, надбавки 162,50 грн, виручка 1 427,50 грн. Цей сценарій має інший набір даних і параметрів, ніж запуск app.py.

Результати роботи

Реалізовано функціональну обробку посилок із відокремленням чистого ядра від I/O. Правила розрахунку можна замінити передаванням інших функцій, не переписуючи алгоритм. Наявні скриншоти підтверджують структуру проєкту, основні частини коду, демонстраційний вивід і успішне проходження тестів.

Висновок

У ході лабораторної роботи розроблено систему доставки для варіанта 6. Обчислення тарифу, надбавок, фільтрації та формування результатів винесено в чисте ядро; введення/виведення залишено в оболонці app.py. Політики прийнятності, тарифу, надбавки й часу передаються як Callable, а обробка створює нові записи замість зміни вхідних. П’ять автоматизованих тестів

пройшли успішно, що підтверджує детермінованість, відсутність мутацій і коректність побудованих політик у перевірених сценаріях.

Додаток:

```python
"""Оболонка побічних ефектів (I/O) для системи доставки."""

from __future__ import annotations

import time

from core import (
    DeliveryReport,
    DeliveryType,
    ParcelInput,
    calculate_base_tariff,
    calculate_surcharge,
    compose,
    make_delivery_processor,
    make_multiplier,
)


def sample_parcels() -> list[ParcelInput]:
    """Повертає тестовий набір вхідних посилок."""
    return [
        {
            "id": 101,
            "weight_kg": 2.5,
            "distance_km": 40.0,
            "delivery_type": "standard",
            "paid": True,
        },
        {
            "id": 102,
            "weight_kg": 18.0,
            "distance_km": 350.0,
            "delivery_type": "express",
            "paid": True,
        },
        {
            "id": 103,
            "weight_kg": 5.0,
            "distance_km": 120.0,
            "delivery_type": "courier",
            "paid": False,
        },
        {
            "id": 104,
            "weight_kg": 45.0,
            "distance_km": 200.0,
            "delivery_type": "standard",
            "paid": True,
        },
        {
            "id": 105,
            "weight_kg": 1.2,
            "distance_km": 15.0,
            "delivery_type": "courier",
            "paid": True,
        },
    ]


def render_report(report: DeliveryReport) -> None:
    """Виводить сформований звіт у консоль (ізольований I/O)."""
    print("=== Звіт системи доставки ===")
    print(f"Оброблено посилок: {report['count']}")
    for p in report["parcels"]:
        print(
            f"  Посилка #{p['id']} [{p['delivery_type']}]: "
            f"вага={p['weight_kg']} кг, відстань={p['distance_km']} км | "
            f"база={p['base_cost']:.2f} грн, "
            f"надбавка={p['surcharge']:.2f} грн, "
            f"разом={p['total_cost']:.2f} грн (ts={p['processed_at']})"
        )
    print(f"Сумарний базовий тариф: {report['total_base_cost']:.2f} грн")
    print(f"Сумарні надбавки:       {report['total_surcharge']:.2f} грн")
    print(f"Загальна виручка:       {report['total_revenue']:.2f} грн")


def run() -> None:
    """Точка входу: налаштовує політики, запускає ядро та друкує звіт."""
    multipliers: dict[DeliveryType, float] = {
        "standard": 1.0,
        "express": 1.5,
        "courier": 1.3,
    }

    def accept_policy(parcel: ParcelInput) -> bool:
        return (
            parcel["paid"]
            and 0 < parcel["weight_kg"] <= 30.0
            and parcel["distance_km"] > 0
        )

    def tariff_policy(parcel: ParcelInput) -> float:
        return calculate_base_tariff(
            parcel,
            rate_per_kg=12.0,
            rate_per_km=1.5,
            type_multipliers=multipliers,
        )

    fuel_adjust = make_multiplier(1.05)

    def add_packaging_fee(surcharge: float) -> float:
        return surcharge + 15.0 if surcharge > 0 else 0.0

    adjusted_surcharge = compose(add_packaging_fee, fuel_adjust)

    def surcharge_policy(base_cost: float, parcel: ParcelInput) -> float:
        raw_surcharge = calculate_surcharge(
            base_cost,
            parcel,
            heavy_threshold_kg=10.0,
            heavy_fee=80.0,
            remote_threshold_km=250.0,
            remote_rate=0.15,
        )
        return adjusted_surcharge(raw_surcharge)

    processor = make_delivery_processor(
        accept=accept_policy,
        tariff_policy=tariff_policy,
        surcharge_policy=surcharge_policy,
        now_fn=time.time,
    )

    parcels = sample_parcels()
    report = processor(parcels)
    render_report(report)


if __name__ == "__main__":
    run()
```

App.py

```python
"""Чисте розрахункове ядро системи доставки (Варіант 6)."""

from __future__ import annotations

from collections.abc import Callable, Iterable, Mapping
from typing import Literal, TypedDict, TypeVar

A = TypeVar("A")
B = TypeVar("B")
C = TypeVar("C")

DeliveryType = Literal["standard", "express", "courier"]


class ParcelInput(TypedDict):
    """Вхідна структура даних посилки (не мутується)."""

    id: int
    weight_kg: float
    distance_km: float
    delivery_type: DeliveryType
    paid: bool


class ProcessedParcel(ParcelInput):
    """Новий запис обробленої посилки з розрахованими полями."""

    base_cost: float
    surcharge: float
    total_cost: float
    processed_at: float | None


class DeliveryReport(TypedDict):
    """Підсумковий звіт обробки партії посилок."""

    count: int
    total_base_cost: float
    total_surcharge: float
    total_revenue: float
    parcels: list[ProcessedParcel]


AcceptFn = Callable[[ParcelInput], bool]
TariffFn = Callable[[ParcelInput], float]
SurchargeFn = Callable[[float, ParcelInput], float]
NowFn = Callable[[], float]


def compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]:
    """Композиція двох функцій: (f ∘ g)(x) = f(g(x))."""
    return lambda x: f(g(x))


def make_multiplier(factor: float) -> Callable[[float], float]:
    """Фабрика функцій-множників для масштабування вартості."""
    return lambda value: value * factor


def calculate_base_tariff(
    parcel: ParcelInput,
    *,
    rate_per_kg: float,
    rate_per_km: float,
    type_multipliers: Mapping[DeliveryType, float],
) -> float:
    """Обчислює базову вартість доставки без використання глобального стану."""
    raw_cost = (
        parcel["weight_kg"] * rate_per_kg
        + parcel["distance_km"] * rate_per_km
    )
    multiplier = type_multipliers[parcel["delivery_type"]]
    return round(raw_cost * multiplier, 2)


def calculate_surcharge(
    base_cost: float,
    parcel: ParcelInput,
    *,
    heavy_threshold_kg: float,
    heavy_fee: float,
    remote_threshold_km: float,
    remote_rate: float,
) -> float:
    """Обчислює суму надбавок за велику вагу та дальню відстань."""
    weight_extra = (
        heavy_fee if parcel["weight_kg"] > heavy_threshold_kg else 0.0
    )
    distance_extra = (
        base_cost * remote_rate
        if parcel["distance_km"] > remote_threshold_km
        else 0.0
    )
    return round(weight_extra + distance_extra, 2)
```

```python

def with_delivery_totals(
    parcel: ParcelInput,
    *,
    base_cost: float,
    surcharge: float,
    total_cost: float,
    processed_at: float | None = None,
) -> ProcessedParcel:
    """Повертає новий словник посилки з результатами без мутації оригіналу."""
    return {
        "id": parcel["id"],
        "weight_kg": parcel["weight_kg"],
        "distance_km": parcel["distance_km"],
        "delivery_type": parcel["delivery_type"],
        "paid": parcel["paid"],
        "base_cost": base_cost,
        "surcharge": surcharge,
        "total_cost": total_cost,
        "processed_at": processed_at,
    }


def filter_by_delivery_type(
    parcels: Iterable[ParcelInput],
    delivery_type: DeliveryType,
) -> list[ParcelInput]:
    """Повертає новий список посилок заданого типу без побічних ефектів."""
    return [p for p in parcels if p["delivery_type"] == delivery_type]


def process_parcels_pure(
    parcels: Iterable[ParcelInput],
    *,
    max_weight_kg: float,
    rate_per_kg: float,
    rate_per_km: float,
    type_multipliers: Mapping[DeliveryType, float],
    heavy_threshold_kg: float,
    heavy_fee: float,
    remote_threshold_km: float,
    remote_rate: float,
) -> DeliveryReport:
    """Чиста функція базової обробки посилок з явно переданими параметрами."""
    processed: list[ProcessedParcel] = []
    total_base = 0.0
    total_surcharge = 0.0
    total_revenue = 0.0

    for parcel in parcels:
        if not parcel["paid"] or not (0 < parcel["weight_kg"] <= max_weight_kg):
            continue
        base_cost = calculate_base_tariff(
            parcel,
            rate_per_kg=rate_per_kg,
            rate_per_km=rate_per_km,
            type_multipliers=type_multipliers,
        )
        surcharge = calculate_surcharge(
            base_cost,
            parcel,
            heavy_threshold_kg=heavy_threshold_kg,
            heavy_fee=heavy_fee,
            remote_threshold_km=remote_threshold_km,
            remote_rate=remote_rate,
        )
        total_cost = round(base_cost + surcharge, 2)
        processed.append(
            with_delivery_totals(
                parcel,
                base_cost=base_cost,
                surcharge=surcharge,
                total_cost=total_cost,
            )
        )
        total_base = round(total_base + base_cost, 2)
        total_surcharge = round(total_surcharge + surcharge, 2)
        total_revenue = round(total_revenue + total_cost, 2)

    return {
        "count": len(processed),
        "total_base_cost": total_base,
        "total_surcharge": total_surcharge,
        "total_revenue": total_revenue,
        "parcels": processed,
    }


def make_delivery_processor(
    *,
    accept: AcceptFn,
    tariff_policy: TariffFn,
    surcharge_policy: SurchargeFn,
    now_fn: NowFn | None = None,
) -> Callable[[Iterable[ParcelInput]], DeliveryReport]:
    """Фабрика вищого порядку, що конфігурує чистий обробник доставки."""

    def process(parcels: Iterable[ParcelInput]) -> DeliveryReport:
        processed: list[ProcessedParcel] = []
        total_base = 0.0
        total_surcharge = 0.0
        total_revenue = 0.0

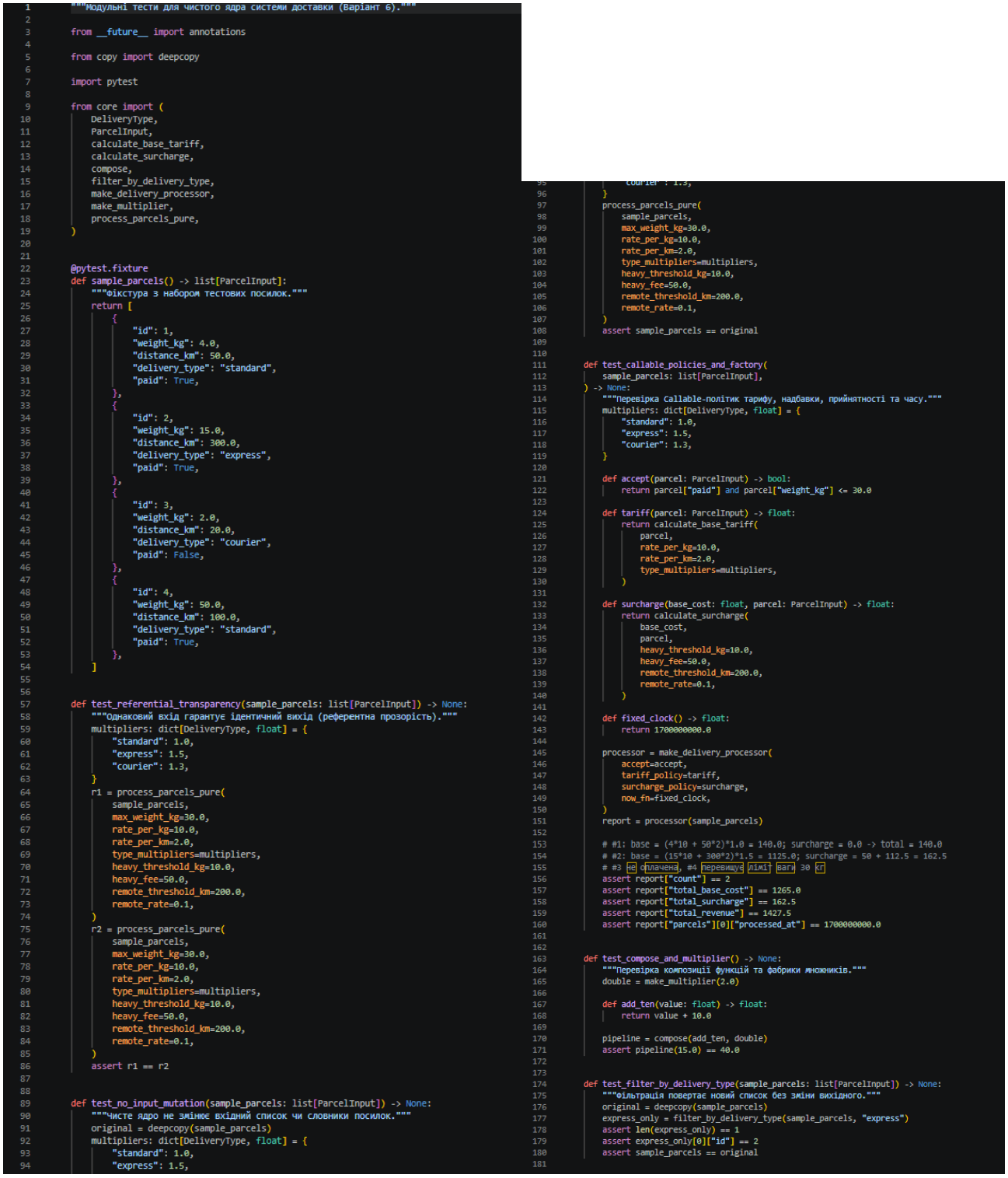
        for parcel in parcels:
            if not accept(parcel):
                continue
            base_cost = round(tariff_policy(parcel), 2)
            surcharge = round(surcharge_policy(base_cost, parcel), 2)
            total_cost = round(base_cost + surcharge, 2)
            timestamp = now_fn() if now_fn is not None else None

            processed.append(
                with_delivery_totals(
                    parcel,
                    base_cost=base_cost,
                    surcharge=surcharge,
                    total_cost=total_cost,
                    processed_at=timestamp,
                )
            )
            total_base = round(total_base + base_cost, 2)
            total_surcharge = round(total_surcharge + surcharge, 2)
            total_revenue = round(total_revenue + total_cost, 2)

        return {
            "count": len(processed),
            "total_base_cost": total_base,
            "total_surcharge": total_surcharge,
            "total_revenue": total_revenue,
            "parcels": processed,
        }

    return process
```

Core.py



```python
"""Модульні тести для чистого ядра системи доставки (Варіант 6)."""

from __future__ import annotations

from copy import deepcopy

import pytest

from core import (
    DeliveryType,
    ParcelInput,
    calculate_base_tariff,
    calculate_surcharge,
    compose,
    filter_by_delivery_type,
    make_delivery_processor,
    make_multiplier,
    process_parcels_pure,
)


@pytest.fixture
def sample_parcels() -> list[ParcelInput]:
    """Фікстура з набором тестових посилок."""
    return [
        {
            "id": 1,
            "weight_kg": 4.0,
            "distance_km": 50.0,
            "delivery_type": "standard",
            "paid": True,
        },
        {
            "id": 2,
            "weight_kg": 15.0,
            "distance_km": 300.0,
            "delivery_type": "express",
            "paid": True,
        },
        {
            "id": 3,
            "weight_kg": 2.0,
            "distance_km": 20.0,
            "delivery_type": "courier",
            "paid": False,
        },
        {
            "id": 4,
            "weight_kg": 50.0,
            "distance_km": 100.0,
            "delivery_type": "standard",
            "paid": True,
        },
    ]


def test_referential_transparency(sample_parcels: list[ParcelInput]) -> None:
    """Однаковий вхід гарантує ідентичний вихід (референтна прозорість)."""
    multipliers: dict[DeliveryType, float] = {
        "standard": 1.0,
        "express": 1.5,
        "courier": 1.3,
    }
    r1 = process_parcels_pure(
        sample_parcels,
        max_weight_kg=30.0,
        rate_per_kg=10.0,
        rate_per_km=2.0,
        type_multipliers=multipliers,
        heavy_threshold_kg=10.0,
        heavy_fee=50.0,
        remote_threshold_km=200.0,
        remote_rate=0.1,
    )
    r2 = process_parcels_pure(
        sample_parcels,
        max_weight_kg=30.0,
        rate_per_kg=10.0,
        rate_per_km=2.0,
        type_multipliers=multipliers,
        heavy_threshold_kg=10.0,
        heavy_fee=50.0,
        remote_threshold_km=200.0,
        remote_rate=0.1,
    )
    assert r1 == r2


def test_no_input_mutation(sample_parcels: list[ParcelInput]) -> None:
    """Чисте ядро не змінює вхідний список чи словники посилок."""
    original = deepcopy(sample_parcels)
    multipliers: dict[DeliveryType, float] = {
        "standard": 1.0,
        "express": 1.5,
        "courier": 1.3,
    }
    process_parcels_pure(
        sample_parcels,
        max_weight_kg=30.0,
        rate_per_kg=10.0,
        rate_per_km=2.0,
        type_multipliers=multipliers,
        heavy_threshold_kg=10.0,
        heavy_fee=50.0,
        remote_threshold_km=200.0,
        remote_rate=0.1,
    )
    assert sample_parcels == original


def test_callable_policies_and_factory(
    sample_parcels: list[ParcelInput],
) -> None:
    """Перевірка Callable-політик тарифу, надбавки, прийнятності та часу."""
    multipliers: dict[DeliveryType, float] = {
        "standard": 1.0,
        "express": 1.5,
        "courier": 1.3,
    }

    def accept(parcel: ParcelInput) -> bool:
        return parcel["paid"] and parcel["weight_kg"] <= 30.0

    def tariff(parcel: ParcelInput) -> float:
        return calculate_base_tariff(
            parcel,
            rate_per_kg=10.0,
            rate_per_km=2.0,
            type_multipliers=multipliers,
        )

    def surcharge(base_cost: float, parcel: ParcelInput) -> float:
        return calculate_surcharge(
            base_cost,
            parcel,
            heavy_threshold_kg=10.0,
            heavy_fee=50.0,
            remote_threshold_km=200.0,
            remote_rate=0.1,
        )

    def fixed_clock() -> float:
        return 1700000000.0

    processor = make_delivery_processor(
        accept=accept,
        tariff_policy=tariff,
        surcharge_policy=surcharge,
        now_fn=fixed_clock,
    )
    report = processor(sample_parcels)

    # #1: base = (4*10 + 50*2)*1.0 = 140.0; surcharge = 0.0 -> total = 140.0
    # #2: base = (15*10 + 300*2)*1.5 = 1125.0; surcharge = 50 + 112.5 = 162.5
    # #3 не оплачена, #4 перевищує ліміт ваги 30 кг
    assert report["count"] == 2
    assert report["total_base_cost"] == 1265.0
    assert report["total_surcharge"] == 162.5
    assert report["total_revenue"] == 1427.5
    assert report["parcels"][0]["processed_at"] == 1700000000.0


def test_compose_and_multiplier() -> None:
    """Перевірка композиції функцій та фабрики множників."""
    double = make_multiplier(2.0)

    def add_ten(value: float) -> float:
        return value + 10.0

    pipeline = compose(add_ten, double)
    assert pipeline(15.0) == 40.0


def test_filter_by_delivery_type(sample_parcels: list[ParcelInput]) -> None:
    """Фільтрація повертає новий список без зміни вихідного."""
    original = deepcopy(sample_parcels)
    express_only = filter_by_delivery_type(sample_parcels, "express")
    assert len(express_only) == 1
    assert express_only[0]["id"] == 2
    assert sample_parcels == original
```

Test_cote.py

Виконання програми

```
C:\Users\Davyd\Downloads\lab01\lab01\.venv\Scripts\python.exe C:\Users\Davyd\Downloads\lab01\lab01\app.py
=== Звіт системи доставки ===
Оброблено посилок: 3
  Посилка #101 [standard]: вага=2.5 кг, відстань=40.0 км | база=90.00 грн, надбавка=0.00 грн, разом=90.00 грн (ts=1794503823.3573005)
  Посилка #102 [express]: вага=18.0 кг, відстань=350.0 км | база=1111.50 грн, надбавка=274.06 грн, разом=1385.56 грн (ts=1794503823.3573146)
  Посилка #105 [courier]: вага=1.2 кг, відстань=15.0 км | база=47.97 грн, надбавка=0.00 грн, разом=47.97 грн (ts=1794503823.357323)
Сумарний базовий тариф: 1249.47 грн
Сумарні надбавки:      274.06 грн
Загальна виручка:      1523.53 грн

Process finished with exit code 0
```

Виконання тесту

Test Re 0 ms

5 tests passed 5 tests total, 0 ms

```
C:\Users\Davyd\Downloads\lab01\lab01\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2025.2.3/plugins/python-ce/helpers/pycharm/_jb_pytest_runner.py
Testing started at 19:17 ...
Launching pytest with arguments C:\Users\Davyd\Downloads\lab01\lab01\tests\test_core.py --no-header --no-summary -q in C:\Users\Davyd\Downloads\lab01\lab01

============================= test session starts =============================
collecting ... collected 5 items

tests/test_core.py::test_referential_transparency PASSED                [ 20%]
tests/test_core.py::test_no_input_mutation PASSED                       [ 40%]
tests/test_core.py::test_callable_policies_and_factory PASSED           [ 60%]
tests/test_core.py::test_compose_and_multiplier PASSED                  [ 80%]
tests/test_core.py::test_filter_by_delivery_type PASSED                 [100%]

============================== 5 passed in 0.02s ==============================

Process finished with exit code 0
```